

Task Plan / Backlog realizacji projektu

Symulator pracy silnika czterosuwowego R4 w OpenGL

Dokument rozбивa PRD na konkretne zadania implementacyjne. Ma sluzyc jako backlog do pracy w zespole oraz jako lista etapow dla Codexa. Kolejnosć jest ustawiona tak, zeby najpierw dowiedzć dzialajace MVP, a dopiero potem dodawac efekty wizualne i bajery.

1. Strategia realizacji

- Nie robimy wszystkiego naraz. Najpierw powstaje stabilny szkielet OpenGL, potem jeden cylinder, potem silnik R4.
- Kazdy etap musi konczyc sie dzialajacym programem, nawet jezeli wizualnie jest jeszcze prosty.
- Codex powinien dostawac zadania etapami: pojedynczy modul, pojedyncza funkcja albo pojedyncza klasa.
- MVP to: wal + 4 tloki + korbowody + zawory + gaz/RPM + kamera + 2 tekstury + 2 swiatla.
- Dopiero po MVP dodajemy HUD, kolory faz, efekty spalania, dym, lepsze modele i polish.

2. Kamienie milowe

ID	Etap	Efekt koncowy
M0	Przygotowanie projektu	Projekt kompiluje sie, okno OpenGL dziala, jest baza pod prace.
M1	Szkielet renderowania	Kamera, shader, mesh, depth test, jeden obiekt 3D.
M2	Jeden cylinder	Jeden cylinder ma tlok, zawory, korbowod i prosty wal.
M3	Silnik R4 MVP	4 cylindry dzialaja z przesunieciami faz, wal obraca sie wspolnie.
M4	Interakcja	Gaz, RPM, pauza, reset, kamera, presety widoku.
M5	Grafika wymagana	Tekstury, dwa swiatla, cieniowanie, normalne.
M6	Efekty i HUD	Panel RPM/faz, kolory suwow, iskra/spaliny opcjonalnie.
M7	Finalizacja	README, czyszczenie kodu, testy, demo do oddania.

3. Backlog zadan

ID	Milestone	Zadanie	Opis	Priorytet	Zaleznosci	Definition of Done
T01	M0	Setup repo/projektu	Ustalic strukture katalogow, build, assets, shaders, src. Sprawdzic kompilacje szkieletu labowego.	Must	-	Projekt startuje bez bledow.
T02	M0	Porzadek w plikach	Oddzielic kod aplikacji od danych: src/, include/, shaders/, textures/, models/.	Should	T01	Czytelna struktura pod dalsze taski.
T03	M1	Klasa Shader	Wrapper do wczytywania	Must	T01	Mozna ustawic P/V/M i

			shaderow i ustawiania uniformow: mat4, vec3, float, int.			tekstury przez metody klasy.
T04	M1	Klasa Camera	Kamera orbitujaca wokol silnika + zoom + presety widoku.	Must	T03	A/D obraca, Q/E zoomuje, 1-4 zmieniaja widok.
T05	M1	Klasa Mesh	VAO/VBO/EBO, struktura Vertex: position, normal, texCoord. Rysowanie przez glDrawElements.	Must	T03	Mesh.draw() rysuje dowolny obiekt.
T06	M1	Proceduralne bryly	Generator cube, cylinder, simple rod. Normale i UV minimum podstawowe.	Must	T05	Na scenie widac kilka bryl z poprawnym depth test.
T07	M1	Transformacje P/V/M	Ujednolicic wysylanie macierzy P, V, M do shaderow.	Must	T03,T04	Kazdy obiekt ma wlasna macierz M.
T08	M2	Model jednego cylindra	Zbudowac obudowe cylindra, tlok i glowice jako proste bryly.	Must	T06,T07	Widoczny pojedynczy cylinder w przekroju.
T09	M2	Ruch tloka	Dodac sinusoidalny ruch tloka na podstawie kata walu.	Must	T08	Tlok porusza sie gora/dol plynnie.
T10	M2	Wal dla jednego cylindra	Dodac obracajacy sie czop/fragment walu.	Must	T09	Element walu obraca sie zgodnie z crankAngle.
T11	M2	Korbowod miedzy punktami	Narysowac rod/cylinder/prostopadlosci an laczacy punkt walu z tlokiem.	Must	T10	Korbowod zawsze laczy tlok i czop walu.
T12	M2	Zawory cylindra	Dodac zawor ssacy i wydechowy, przesuwane zaleznie od suwu.	Must	T09	Na ssaniu otwiera sie ssacy, na wydechu wydechowy.
T13	M2	Logika suwow 720 stopni	Enum StrokeType i funkcja okreslajaca suw po fazie 0-720.	Must	T09	Dla faz 0-180/180-360/360-540/540-720 zwracany jest poprawny suw.
T14	M3	Klasa Cylinder	Zamknac logike jednego cylindra w klasie: update(), draw(), phaseOffset.	Must	T08-T13	Mozna stworzyc wiele cylindrow z roznymi fazami.
T15	M3	Silnik R4	Stworzyc 4 instancje Cylinder w jednej klasie EngineSimulator.	Must	T14	Cztery cylindry pracuja w jednej scenie.
T16	M3	Wspolny wal korbowy	Dodac wspolny wal przez cala dlugosc silnika + czopy przesuniete fazowo.	Must	T15	Wal obraca sie jako jeden mechanizm.
T17	M3	Blok silnika w przekroju	Dodac blok/glowice tak, aby nie zaslanialy mechaniki.	Should	T15	Widac, ze to silnik, a ruchome elementy nie sa zakryte.
T18	M3	Kolo zamachowe/pasowe	Dodac kolo na koncu walu obracajace sie razem z nim.	Should	T16	Kolo obraca sie z walem.
T19	M4	Throttle/RPM	Dodac gaz od 0 do 1 i plynne dochodzenie RPM do targetRpm.	Must	T15	W/S zmienia obroty, animacja przyspiesza/zwalnia.
T20	M4	Pauza i reset	SPACE pauza/wznowienie, R reset kata i RPM.	Must	T19	Program da sie zatrzymac i zresetowac.
T21	M4	Sterowanie kamera	Dopiac wszystkie klawisze	Must	T04	Mozna wygodnie pokazac

			kamery i presety.			silnik z roznnych stron.
T22	M5	Texture loader	Wczytywanie tekstur przez lodepng/stb_image i bindowanie do shaderow.	Must	T05	Minimum jedna tekstura dziala na obiekcie.
T23	M5	Teksturowanie obiektow	Przypisac minimum 2 tekstury: metal/dark_metal/rubber/gauge.	Must	T22	Glowna scena nie jest jednokolorowa.
T24	M5	Normalne dla bryl	Poprawic normale proceduralnych meshy.	Must	T06	Swiatlo reaguje poprawnie na ksztalty.
T25	M5	Shader Phong/Blinn-Phong	Dodac ambient, diffuse, specular.	Must	T23,T24	Widoczne cienie i blyski na metalu.
T26	M5	Dwa zrodla swiatla	Swiatlo kierunkowe + punktowe nad silnikiem.	Must	T25	Na obiektach widac wplyw dwoch swiatel.
T27	M6	HUD/panel 3D	Pokazac RPM, throttle i fazy cylindrow.	Should	T19,T13	Uzytkownik widzi stan silnika.
T28	M6	Kolory faz pracy	Opcja F: kolorowe podswietlanie suwow.	Should	T13,T27	Fazy sa czytelne podczas demonstracji.
T29	M6	Efekt spalania	Krotki blysk w cylindrze w suwie pracy.	Could	T13,T15	W suwie pracy pojawia sie widoczny efekt.
T30	M6	Spaliny/dym	Proste czasteczki lub przezroczyste kulki przy wydechu.	Could	T13,T15	W suwie wydechu widac efekt spalin.
T31	M7	README	Opis projektu, sterowanie, wymagania, uruchomienie, autorzy.	Must	MVP	Jest gotowa instrukcja dla prowadzacego.
T32	M7	Czyszczenie i komentarze	Usunac smieci, dodac komentarze przy waznej matematyce.	Must	MVP	Kod jest gotowy do obrony.
T33	M7	Test oddania	Sprawdzic na czysto: build, assets, sciezki, uruchomienie, sterowanie.	Must	T31,T32	Projekt gotowy do pokazania.

4. Proponowany podzial pracy

Rola	Taski	Odpowiedzialnosc
Osoba A - Core/OpenGL	T01-T07, T22-T26	Szkielet, shadery, meshe, tekstury, swiatla, render pipeline.
Osoba B - Mechanika silnika	T08-T21	Cylinder, tloki, wal, korbowody, zawory, R4, throttle/RPM.
Wspolne	T27-T33	HUD, efekty, README, testy, finalizacja i przygotowanie demo.

5. Kolejnosc pracy z Codexem

Krok	Polecenie dla Codexa
1	Popros Codexa o klase Shader i minimalny main z P/V/M oraz depth test.
2	Dodaj klase Mesh i generator cube/cylinder. Nie ruszaj jeszcze mechaniki.
3	Zrob jeden cylinder statyczny: obudowa + tlok + dwa zawory.
4	Dodaj crankAngle i ruch tloka w jednym cylindrze.

5	Dodaj korbowod miedzy dwoma punktami.
6	Dodaj logike czterosuwu 720 stopni i ruch zaworow.
7	Zrefaktoryzuj do klasy Cylinder i skopiuj na 4 cylindry.
8	Dodaj EngineSimulator z throttle, rpm, pause, reset.
9	Dodaj tekstury i dwa swiatla.
10	Dodaj HUD/panel, kolory faz, finalne efekty i README.

6. Gotowe prompty do Codexa

Prompt ogolny na start

Pracujemy nad projektem C++ OpenGL/GLM: interaktywny symulator pracy silnika czterosuwowego R4. Korzystamy z nowoczesnego OpenGL, VAO/VBO/EB0, shaderow GLSL, glDrawArrays/glDrawElements oraz transformacji GLM. Nie wolno uzywac starego OpenGL. Realizuj zadania etapami i nie przebudowuj calego projektu naraz.

Prompt do taska

Zrealizuj tylko task [ID]. Nie zmieniaj niepowiazanych plikow. Po zmianach wypisz: jakie pliki zmieniles, jak przetestowac, jakie sa znane ograniczenia.

Prompt do debugowania

Przeanalizuj blad kompilacji/runtime. Nie przepisuj calego projektu. Wskaz minimalna poprawke i pokaz konkretny diff/kod do zmiany.

Prompt do refaktoru

Zrefaktoryzuj obecny kod do klas wskazanych w PRD, ale zachowaj dzialanie. Najpierw pokaz plan zmian, potem wykonaj minimalny bezpieczny refaktor.

7. Checklist finalny przed oddaniem

- ☐ Projekt kompiluje sie na docelowym srodowisku.
- ☐ Program startuje bez koniecznosci poprawiania sciezek.
- ☐ Widac silnik R4 w przekroju.
- ☐ Wal, tloki, korbowody i zawory sa animowane.
- ☐ W/S zmieniaja RPM, SPACE pauzuje, R resetuje.
- ☐ Kamera ma presety i sterowanie.
- ☐ Sa minimum 2 tekstury.
- ☐ Sa minimum 2 zrodla swiatla.

- ☐ Cieniowanie jest widoczne.
- ☐ Nie ma zakazanych funkcji starego OpenGL.
- ☐ Nie ma koloru różowego.
- ☐ Jest README i opis sterowania.
- ☐ Każdy członek zespołu rozumie swoją część kodu.

8. Zasada dowożenia

Po każdym tasku projekt ma dalej działać. Jeżeli task zaczyna rozwalac build, należy go cofnąć albo uprościć. Najpierw dowozimy MVP, potem efekty. Ładny, ale niedziałający silnik jest gorszy niż prosty silnik, który płynnie działa i da się obronić.